

rapira26 VM — Bytecode Reference

Source of truth: `vm-core/src/decoder.rs`

1. Encoding Conventions

Each instruction begins with a single **opcode byte**. The upper nibble (bits 7–4) identifies the instruction group; the lower nibble (bits 3–0) selects the variant. Multi-byte operands immediately follow the opcode and are always encoded as little-endian 32-bit signed integers (**i32le**, 4 bytes), unless otherwise noted.

Notation used below

i32le	4-byte little-endian signed integer operand
f64le	8-byte little-endian IEEE 754 double
u8	1-byte unsigned integer
—	no operands (opcode byte only)

2. Instruction Reference

2.1. Group **0x0_** — Miscellaneous / Binary Operations

Opcode	Mnemonic	Operands	Description
0x00	NOP	—	No operation.
0x01	BINOP ADD	—	Pop two integers, push their sum.
0x02	BINOP SUB	—	Pop two integers, push their difference (TOS-1 – TOS).
0x03	BINOP MUL	—	Pop two integers, push their product.
0x04	BINOP DIV	—	Pop two numbers and divide TOS-1 by TOS; exact integer division stays integer, otherwise push a real.
0x05	BINOP MOD	—	Pop two integers, push remainder.
0x06	BINOP LT	—	Push 1 if TOS-1 < TOS, else 0.
0x07	BINOP LEQ	—	Push 1 if TOS-1 ≤ TOS, else 0.
0x08	BINOP GT	—	Push 1 if TOS-1 > TOS, else 0.
0x09	BINOP GEQ	—	Push 1 if TOS-1 ≥ TOS, else 0.
0x0a	BINOP EQ	—	Push 1 if TOS-1 == TOS, else 0.
0x0b	BINOP NEQ	—	Push 1 if TOS-1 ≠ TOS, else 0.
0x0c	BINOP AND	—	Push 1 if both TOS-1 and TOS are non-zero.
0x0d	BINOP OR	—	Push 1 if either TOS-1 or TOS is non-zero.
0x0e	BINOP IDIV	—	Pop two integers and push their quotient rounded down.

Opcode	Mnemonic	Operands	Description
0x0f	BINOP POW	—	Pop base and exponent and push base raised to exponent.

The BINOP lower nibble encodes the operator (0x01–0x0f).

2.2. Group 0x1_ — Stack / Constants / Control

Opcode	Mnemonic	Operands	Description
0x10	CONST	i32le value	Push immediate integer value onto the stack.
0x11	STRING	i32le index	Push the index -th string from the string table.
0x12	TUPLE	i32le n	Pop n values from the stack, pack them into a tuple, push the tuple.
0x13	STS	—	Store to slice: pop slice, then pop value and assign into the slice.
0x14	STA	—	Store to aggregate: pop aggregate, pop index, pop value; assign value at index into aggregate .
0x15	JMP	i32le offset	Unconditional jump to offset (absolute position in code section).
0x16	END	—	End of procedure: return top-of-stack value to caller.
0x17	SLICE	u8 bounds	Push a slice (view) of an aggregate. bounds is a bitmask: bit 0 set means from is on the stack, bit 1 set means to is on the stack. Thus 0 = [:], 1 = [from:], 2 = [:to], 3 = [from:to].
0x18	DROP	—	Discard (pop) the top value from the stack.
0x19	DUP	—	Duplicate the top value of the stack.
0x1a	SWAP	—	Swap the two topmost values on the stack.
0x1b	ELEM	—	Pop aggregate and index, push the element at that index.
0x1c	CONSTF	f64le value	Push immediate 64-bit floating-point value (little-endian IEEE 754) onto the stack.

Opcodes 0x1d–0x1f are unused.

2.3. Group 0x2_ — Load

Load a value onto the stack from a variable slot. The lower nibble encodes the scope (**value relation**).

Opcode	Mnemonic	Operands	Description
0x21	LOAD global	i32le index	Push global variable at slot index .
0x22	LOAD local	i32le index	Push local variable at slot index .
0x23	LOAD arg	i32le index	Push function argument at position index .
0x24	LOAD capture	i32le index	Push captured (closure) variable at slot index .

Lower nibble: 0x1 = Global, 0x2 = Local, 0x3 = Arg, 0x4 = Capture. Opcodes 0x20, 0x25–0x2f are unused.

2.4. Group 0x3_ — Unary Operations

Unary operations on the top-of-stack value.

Opcode	Mnemonic	Operands	Description
0x30	UNARY Negate	—	Pop a value, push its arithmetic negation ($-TOS$).
0x31	UNARY Not	—	Pop a value, push 1 if $TOS == 0$ else 0 (logical not).

Lower nibble: 0x0 = Negate, 0x1 = Not. Opcodes 0x32–0x3f are unused.

2.5. Group 0x4_ — Store

Store the top-of-stack value into a variable slot. Lower nibble encodes scope, same as **LOAD**.

Opcode	Mnemonic	Operands	Description
0x41	STORE global	i32le index	Pop TOS and store into global variable at slot index .
0x42	STORE local	i32le index	Pop TOS and store into local variable at slot index .
0x43	STORE arg	i32le index	Pop TOS and store into function argument slot index .
0x44	STORE capture	i32le index	Pop TOS and store into captured variable at slot index .

Opcodes 0x40, 0x45–0x4f are unused.

2.6. Group 0x5_ — Control Flow / Procedure Management / User Types

Opcode	Mnemonic	Operands	Description
0x50	CJMP iszero	i32le offset	Jump to offset if $TOS == 0$; pop TOS.
0x51	CJMP isnonzero	i32le offset	Jump to offset if $TOS \neq 0$; pop TOS.
0x52	BEGIN	i32le args, i32le locals	Start of procedure with args arguments and locals local variables. Initialises locals to empty. Cannot use captured variables.
0x53	CBEGIN	i32le args, i32le locals	Start of closure body with args arguments and locals locals. May reference captured variables.
0x54	CLOSURE	i32le offset, i32le arity	Construct a closure whose bytecode starts at offset and has arity .
0x55	CALLC	i32le arity	Call the closure on $TOS-(arity)$ with arity arguments; push the return value.

Opcode	Mnemonic	Operands	Description
0x56	CALL	i32le offset, i32le n	Call the function whose bytecode starts at offset with n arguments; push the return value.
0x58	ARRAY	i32le n	Test whether TOS is an array of exactly n elements.
0x59	VARIANT	i32le schema	Construct a user-defined variant using schema schema . Pop the number of field values declared by the schema, then push the constructed variant.
0x5a	LINE	i32le n	Annotation: following bytecode corresponds to source line n . Used for diagnostics only.
0x5b	VARIANTTAG	—	Pop a user-defined variant and push its numeric tag.
0x5c	FIELD	i32le index	Pop a user-defined variant and push its zero-based field index .
0x5d	SETFIELD	i32le index	Pop a variant, then a replacement value; assign the value to the variant's zero-based field index .
0x5e	ISVARIANT	i32le schema	Pop a value and push whether it is an instance of variant schema schema .

schema is a zero-based index into the bytefile's variant-schema table. A schema records the variant tag and its field names; it is emitted from a Rapira `тип` definition. **FIELD** and **SETFIELD** use the schema's zero-based field ordering.

Opcodes 0x57 and 0x5f are unused.

2.7. Group 0x6_ — (unused)

All opcodes 0x60–0x6f are unused.

2.8. Group 0x7_ — Builtin Calls / Bool / Null

Opcode	Mnemonic	Operands	Description
0x70	CALLBUILTIN Lread	i32le n	Call built-in Lread . n packs the typed/untyped flag and argument count.
0x71	CALLBUILTIN Lwrite	i32le n	Call built-in Lwrite . n packs the newline flag and argument count; the newline flag is bit 1 << 30.
0x73	CALLBUILTIN Tbool	—	Call built-in Tbool .
0x74	CALLBUILTIN Ttext	—	Call built-in Ttext .
0x75	BOOL	u8	Push a boolean value onto the stack. Operand: non-zero = true , zero = false .
0x76	NULL	—	Push the null value.

Opcode	Mnemonic	Operands	Description
0x77	CALLBUILTIN Abs	—	Call built-in Abs.
0x78	CALLBUILTIN Ttuple	—	Call built-in Ttuple.
0x79	CALLBUILTIN Sqrt	—	Call built-in Sqrt.
0x7a	CALLBUILTIN Floor	—	Call built-in Floor.
0x7b	CALLBUILTIN Round	—	Call built-in Round.
0x7c	CALLBUILTIN Tslice	—	Call built-in Tslice.
0x7d	CALLBUILTIN Tint	—	Call built-in Tint.
0x7e	CALLBUILTIN Tfloat	—	Call built-in Tfloat.
0x7f	CALLBUILTIN Tnull	—	Call built-in Tnull.

Opcode 0x72 is unused. The decoder assigns zero to `n` for all built-ins other than `Lread` and `Lwrite`.

3. Summary Table

All defined opcode bytes at a glance. “Operand bytes” is the total number of additional bytes the instruction consumes beyond the opcode byte itself.

Hex	Mnemonic	Operand bytes
0x00	NOP	0
0x01	BINOP ADD	0
0x02	BINOP SUB	0
0x03	BINOP MUL	0
0x04	BINOP DIV	0
0x05	BINOP MOD	0
0x06	BINOP LT	0
0x07	BINOP LEQ	0
0x08	BINOP GT	0
0x09	BINOP GEQ	0
0x0a	BINOP EQ	0
0x0b	BINOP NEQ	0
0x0c	BINOP AND	0
0x0d	BINOP OR	0
0x0e	BINOP IDIV	0
0x0f	BINOP POW	0
0x10	CONST	4
0x11	STRING	4
0x12	TUPLE	4
0x13	STS	0

Hex	Mnemonic	Operand bytes
0x14	STA	0
0x15	JMP	4
0x16	END	0
0x17	SLICE	1
0x18	DROP	0
0x19	DUP	0
0x1a	SWAP	0
0x1b	ELEM	0
0x1c	CONSTF	8
0x21	LOAD global	4
0x22	LOAD local	4
0x23	LOAD arg	4
0x24	LOAD capture	4
0x30	UNARY Negate	0
0x31	UNARY Not	0
0x41	STORE global	4
0x42	STORE local	4
0x43	STORE arg	4
0x44	STORE capture	4
0x50	CJMP iszero	4
0x51	CJMP isnonzero	4
0x52	BEGIN	8
0x53	CBEGIN	8
0x54	CLOSURE	8
0x55	CALLC	4
0x56	CALL	8
0x58	ARRAY	4
0x59	VARIANT	4
0x5a	LINE	4
0x5b	VARIANTTAG	0
0x5c	FIELD	4
0x5d	SETFIELD	4
0x5e	ISVARIANT	4
0x70	CALLBUILTIN Lread	4
0x71	CALLBUILTIN Lwrite	4
0x73	CALLBUILTIN Tbool	0

Hex	Mnemonic	Operand bytes
0x74	CALLBUILTIN Ttext	0
0x75	BOOL	1
0x76	NULL	0
0x77	CALLBUILTIN Abs	0
0x78	CALLBUILTIN Ttuple	0
0x79	CALLBUILTIN Sqrt	0
0x7a	CALLBUILTIN Floor	0
0x7b	CALLBUILTIN Round	0
0x7c	CALLBUILTIN Tslice	0
0x7d	CALLBUILTIN Tint	0
0x7e	CALLBUILTIN Tfloat	0
0x7f	CALLBUILTIN Tnull	0